

AI-Powered Job Portal – Project Summary

1 . Project Overview

2 . Project Name: AI-Powered Job Portal

3 . Description: AI-based job portal that allows recruiters to post jobs and candidates to apply with AI-powered resume matching feature.

4 . Goal: Build a full-stack application with microservices architecture, secure authentication (JWT), resume analysis, and React frontend.

5 . Architecture

Microservices: 1 . Auth Service – User registration, login, JWT generation. 2 User Service – Profile management, admin user management. 3. Job Service – Post/update/delete jobs, search, pagination. 4 . Application Service – Apply jobs, upload resumes, track applications. 5 AI Service – Resume extraction, skill matching, AI suggestions. 6 . API Gateway – Routes requests to appropriate services.

High-Level Flow:

React Frontend | ↓ API Gateway |

| | | | Auth User Job Application AI Service Service Service Service Service | MySQL

1 . Technology Stack

Frontend: ReactJS, JavaScript, Axios, HTML, CSS Backend: Spring Boot, Spring Security, JWT, Hibernate Database: MySQL Microservices: Spring Cloud Gateway AI Integration: Spring AI Version Control: GitHub

1 . User Roles

ADMIN: Manage users, view all jobs and applications. RECRUITER: Post/update jobs, view applications. CANDIDATE: Register/login, search jobs, apply, upload resume, view AI match score.

1 . Security

Uses JWT for authentication. Token sent in Authorization header as Bearer <token>. Spring Security validates token and grants access based on role.

1 . Database Design (Basic Overview)

Users Table – id, name, email, password, role Jobs Table – id, title, description, skills_required, location Applications Table – id, user_id, job_id, resume_path, status

Relationships: One user → many applications One job → many applications

1 . API Structure (Examples)

Auth: POST /auth/register POST /auth/login

Jobs: GET /jobs POST /jobs PUT /jobs/{id} DELETE /jobs/{id}

Applications: POST /applications/apply GET /applications/user/{id}

1 . AI Workflow

2 . Candidate uploads resume

3 . AI service extracts text

4 . Compares with job requirements

5 . Returns matching percentage and skill suggestions

6 . Frontend Communication

Frontend uses Axios to send HTTP requests. JWT stored in localStorage and sent in headers for endpoints.

```
Example: axios.get("/jobs", { headers: { Authorization: "Bearer " + token } }) .then(response => console.log(response.data));
```

1 . Future Enhancements

Docker containerization of services Cloud deployment (AWS, Azure) CI/CD pipeline setup Email notification system Event-driven architecture using Kafka